

ML 24/25-02 Investigate Input Reconstruction by using Classifiers

Priyanshu Bandyopadhyay
priyanshu.bandyopadhyay@stud.fra-a-uas.de

Manoj Hanumanthu
manoj.hanumanthu@stud.fra-uas.de

Akshay Ganesh Gudekar
akshay.gudekar@stud.fra-uas.de

Abstract - *This study dives deep into the world of Sparse Distributed Representations (SDRs). We reconstruct inputs from SDRs by using two classifiers: the Hierarchical Temporal Memory (HTM) classifier and the K-Nearest Neighbors (KNN) classifier and compare them. By using similarity metrics, our work unveils how closely the predicted values align with the encoded forms and how accurately the classifiers can reconstruct back the original inputs. Lively visualizations and rigorous analysis spotlight the strengths and nuances of each classifier, revealing insights that could revolutionize data reconstruction in diverse applications. Our findings will enrich the understanding of SDR-based classification and open up new horizons in intelligent data processing.*

Keywords: *Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), K-Nearest Neighbors (KNN), Machine Learning, Data Reconstruction, Classifier Performance, Input Encoding, Similarity Metrics, Pattern Recognition.*

I. INTRODUCTION

Reconstructing input data from encoded representations is a significant challenge in machine learning, especially when using Sparse Distributed Representations (SDRs). SDRs are inspired by the human neocortex and are characterized by their sparsity and distributed activation patterns, which contribute to their robustness in pattern recognition and efficient storage of high-dimensional data [1]. The ability to reconstruct original inputs from SDRs plays a crucial role in various areas of artificial intelligence, particularly in data compression, error correction, and predictive modeling.

Hierarchical Temporal Memory (HTM) is a machine learning model designed to process and learn temporal patterns based on SDRs. HTM mimics the structure of the neocortex by utilizing a biologically plausible learning algorithm that identifies sequences in streaming data. The model is particularly effective at learning time-dependent patterns and making predictions without requiring traditional backpropagation-based training [2]. HTM achieves this by employing a Spatial Pooler (SP) to convert input data into SDRs and a Temporal Memory (TM) module to recognize sequences. The inherent sparsity of SDRs in HTM ensures that the system maintains noise tolerance and generalization capabilities.

On the other hand, K-Nearest Neighbors (KNN) is a classical machine learning algorithm that classifies data based on proximity to previously seen examples. Unlike HTM, which builds a hierarchical representation of patterns, KNN relies on simple distance metrics, such as Euclidean distance, to compare input data points to their nearest neighbors [3]. Despite its simplicity, KNN remains an effective baseline for evaluating classification and reconstruction tasks due to its ability to retain local data structure without requiring extensive training.

Through this research, we aim to compare the effectiveness of SDR-based learning techniques and traditional distance-based classifiers in data reconstruction tasks. By analyzing the reconstruction performance of HTM and KNN Classifiers, we seek to understand their respective advantages and limitations in retaining and reconstructing structured data patterns. The insights from this study may contribute to the development of improved reconstruction methods in future machine learning applications.

II. LITERATURE REVIEW

The reconstruction of input data from encoded representations is a critical challenge in artificial intelligence and machine learning. Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and K-Nearest Neighbors (KNN) have been widely used for classification and reconstruction tasks due to their ability to recognize and learn patterns effectively. Our study focuses on leveraging these methods to investigate input reconstruction using classifiers. This section reviews relevant literature on SDR-based learning, HTM's role in sequence modeling, KNN's effectiveness in classification and reconstruction.

A. Sparse Distributed Representations (SDRs) in Learning and Reconstruction

Sparse Distributed Representations (SDRs) form the foundation of HTM-based learning systems. SDRs are biologically inspired encodings that represent information using sparse, high-dimensional binary vectors. Hawkins and Ahmad [1] introduced SDRs as an essential component for neocortical learning, highlighting their advantages in robustness, noise tolerance, and capacity for efficient information storage. Their research demonstrated how SDRs

allow neural networks to process patterns with minimal information loss, making them well-suited for classification and reconstruction tasks.

In a more recent study, Cui, Ahmad, and Hawkins [4] explored how SDRs contribute to classification accuracy and generalization. They showed that SDRs improve the performance of learning algorithms by providing distributed representations that retain essential input features. Their work validated the effectiveness of SDRs in reconstructing input data from sparse encodings, which aligns with our research objective of utilizing SDRs for input reconstruction through classifiers.

B. Hierarchical Temporal Memory (HTM) for Pattern Recognition and Reconstruction

Hierarchical Temporal Memory (HTM) is an advanced sequence-learning algorithm that mimics the neocortex's ability to recognize spatial and temporal patterns. HTM leverages SDRs to encode input data, followed by spatial pooling and temporal memory mechanisms to extract and predict patterns. In their seminal work, Hawkins, Ahmad, and Dubinsky [2] presented HTM's core principles, demonstrating its effectiveness in learning sequences and making real-time predictions. Their research laid the groundwork for HTM-based classification and reconstruction models, proving that SDR encoding and temporal learning significantly enhance pattern recognition.

Further research by Cui et al. [4] investigated HTM's ability to reconstruct missing or distorted input patterns. They demonstrated that HTM models can generate high-accuracy predictions by continuously learning from streaming input data. This aligns with our approach, where HTM is trained on a dataset, and its predictive capability is evaluated using similarity metrics between the reconstructed and original inputs. A critical advantage of HTM over traditional machine learning models is its ability to learn incrementally without requiring retraining on the entire dataset. This characteristic makes HTM particularly useful in real-time applications where continuous learning is necessary. Given these properties, HTM serves as a robust classifier in our study, capable of reconstructing input data based on learned spatial and temporal relationships.

C. K-Nearest Neighbors (KNN) for Classification and Reconstruction

The K-Nearest Neighbors (KNN) algorithm is a widely used non-parametric classification method that employs distance-based similarity measures to classify patterns or reconstruct inputs. Introduced by Cover and Hart, KNN operates on the principle that data points with similar features are likely to belong to the same class or value, establishing it as a robust method for pattern recognition tasks [3]. Its simplicity and theoretical guarantees, such as asymptotic error bounds relative to the Bayes optimal classifier, make it a strong

candidate for input reconstruction in scenarios requiring minimal assumptions about data distributions [3].

A key advantage of KNN is its ability to handle non-linear decision boundaries, making it particularly effective for reconstructing data encoded in Sparse Distributed Representations (SDRs). For example, research by Beretta and Santaniello demonstrated KNN's utility in estimating missing values in datasets while preserving local structural relationships, achieving optimal accuracy with $k=3$ neighbors and F-based feature filtering [5]. These findings support our use of KNN as a baseline for evaluating reconstruction accuracy alongside Hierarchical Temporal Memory (HTM).

III. METHODOLOGY

A. Overall Experiment Flow

The flow is designed to achieve input reconstruction through a structured way, involving Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and K-Nearest Neighbors (KNN) classifiers. The flow diagram is given in Fig. 1. The process begins with input encoding, where scalar inputs are transformed into SDRs using the Scalar Encoder from the NeocortexAPI repository. The scalar encoder produces SDRs. These SDRs are then passed through a Spatial Pooler (SP), which learns to generate stable and robust SDR representations. The stable SDRs serve as the foundation for training both HTM Classifier and KNN Classifier, which are subsequently employed to reconstruct the original input values from test data.

The reconstruction process involves decoding the SDRs back into their original form using the classifiers. Once the input values are reconstructed, they are compared with the original input using similarity metrics. The results are visualized using the ScottPlot library in .NET, providing a clear and intuitive representation of the system's performance. This architecture highlights the effectiveness of combining HTM and KNN classifiers for input reconstruction, offering valuable insights for applications in predictive modeling and data compression. The primary objective is to evaluate the ability of HTM and KNN classifiers to accurately reconstruct inputs, focusing on faithfully recreating the structure of the original encoded data.

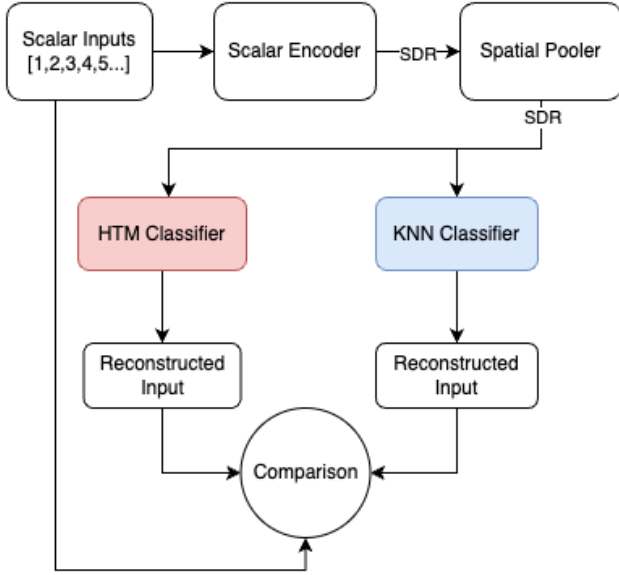


Fig. 1: Experiment Flow Diagram

B. Input Encoding

The Scalar Encoder is used to transform numerical input values into Sparse Distributed Representations (SDRs).

1) Steps for Input Encoding:

- A numerical array is selected as input.
- The input values are fed into the Scalar Encoder from the NeocortexAPI repository.
- The encoder converts each scalar value into a binary SDR representation, where active bits correspond to the input's position within a defined range.

2) Encoder Configuration Parameters:

- W (Width of active bits): 21
- N (Total bits in encoding): 200
- MinVal and MaxVal: Defines the range of input values (1 to max value)
- Clip Input: False (Allows values outside the defined range)
- Radius: -1 (Controls the degree of overlap between encodings)

The encoded SDRs serve as the foundation for training the HTM and KNN classifiers.

C. SDR Generation

After encoding, the Spatial Pooler (SP) processes the SDRs to learn stable representations of input patterns. The SP is a key component of Hierarchical Temporal Memory (HTM), responsible for boosting, inhibition, and learning synaptic connections to ensure SDR sparsity and robustness.

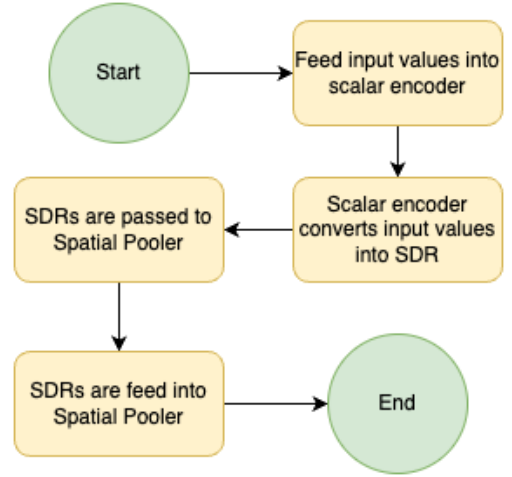


Fig. 2: Flowchart of robust and stable SDR generation

As shown in Fig. 2 the encoded Sparse Distributed Representations (SDRs) are fed into the Spatial Pooler (SP), a core component of the Hierarchical Temporal Memory (HTM) framework. The SP processes the input patterns and activates specific columns based on the learned synaptic connections, which are strengthened over multiple learning cycles. This process ensures the generation of stable SDRs, where similar input patterns activate similar columns, maintaining consistency and robustness in the representations. Once the SP has been trained and stable SDRs are produced, these representations are used for both training and testing the classifiers. The trained SP output serves as the foundation for the classifiers to learn and later reconstruct the original input values, enabling accurate and efficient input reconstruction.

Spatial Pooler Configuration Parameters:

- CellsPerColumn: 10
- NumColumns: 1024
- MaxBoost: 5.0
- DutyCyclePeriod: 100
- MinPctOverlapDutyCycles: 1.0
- NumActiveColumnsPerInhArea: 2% of total columns?
- PotentialRadius: 15% of input bits

The trained Spatial Pooler ensures that SDR representations remain stable over time, forming a basis for classification and reconstruction.

D. Classification and Reconstruction

Once SDRs are generated, they are used to train the HTM and KNN classifiers, which learn to map SDR patterns to original input values. After training, these classifiers attempt to reconstruct test inputs.

1) HTM Classifier Implementation:

The HTM Classifier operates by learning associations between Sparse Distributed Representations (SDRs) and their corresponding input values. During the training phase, it builds a temporal memory that enables it to recognize and

recall previously encountered patterns. This temporal memory is crucial for maintaining context and continuity in sequential data. In the testing phase, the classifier leverages this learned knowledge to predict the most likely input value based on the activation patterns of the SDRs. By analyzing the SDRs, the HTM Classifier can effectively reconstruct the original input values, demonstrating its ability to generalize from learned patterns and accurately predict unseen data. This capability makes the HTM Classifier a powerful tool for tasks such as input reconstruction, anomaly detection, and predictive modeling.

2) KNN Classifier Implementation:

The KNN Classifier stores Sparse Distributed Representations (SDRs) in memory, allowing it to compare and analyze patterns during the reconstruction process. When reconstructing inputs, the classifier identifies the nearest neighbors in the SDR space by calculating the similarity between the input SDR and the stored representations. The K value, which determines the number of neighbors considered for prediction, is carefully optimized to balance accuracy and computational efficiency. By selecting the most similar SDRs and averaging their corresponding input values, the KNN Classifier effectively reconstructs the original inputs. This approach leverages the simplicity and effectiveness of the KNN algorithm, making it a reliable method for input reconstruction tasks.

3) Reconstruction Process:

The reconstruction process (Fig. 3) begins by randomly shuffling the input dataset to ensure an unbiased distribution of data. The shuffled dataset is then split into two subsets: 80% for training and 20% for testing.

Why are we splitting the input data?

To properly evaluate our classifiers, we split the dataset into two parts—80% for training and 20% for testing. Initially, we tried training the classifiers on the entire dataset and then reconstructing the inputs. The results were perfect, with 100% similarity, but that didn't tell us much. There were no differences to analyze, no way to see how well the classifiers were performing.

So, we decided to divide the data for two main reasons:

a) Following a Standard Machine Learning Practice

Typically, in machine learning, you train a model on most of the data (80%) and test it on the remaining (20%) to see how well it generalizes. This helps check whether the model has learned patterns or is just memorizing the data.

b) Testing Robustness to Small Changes

In real-world scenarios, encoded data might not always be perfect—it can get slightly corrupted or altered. By training the classifiers on one set of SDRs (Sparse Distributed Representations) and then testing them on a different set, we can see how well they handle minor variations. Can they still reconstruct the inputs correctly? This is an important question because, in practical applications, data isn't always clean.

To further verify their performance, we also tested reconstruction using the same data they were trained on. As expected, both classifiers performed perfectly when dealing with familiar SDRs, which confirms that they work well under ideal conditions.

Here's how the process works: The training subset is encoded into SDRs, which are then used to train the HTM and KNN classifiers. Once they're trained, we feed them SDR-encoded test data and ask them to reconstruct the original inputs. Finally, we compare the reconstructed values with the actual inputs to measure how accurately they recover the original data. This gives us a clear picture of how well the classifiers perform and whether they can handle real-world variations.

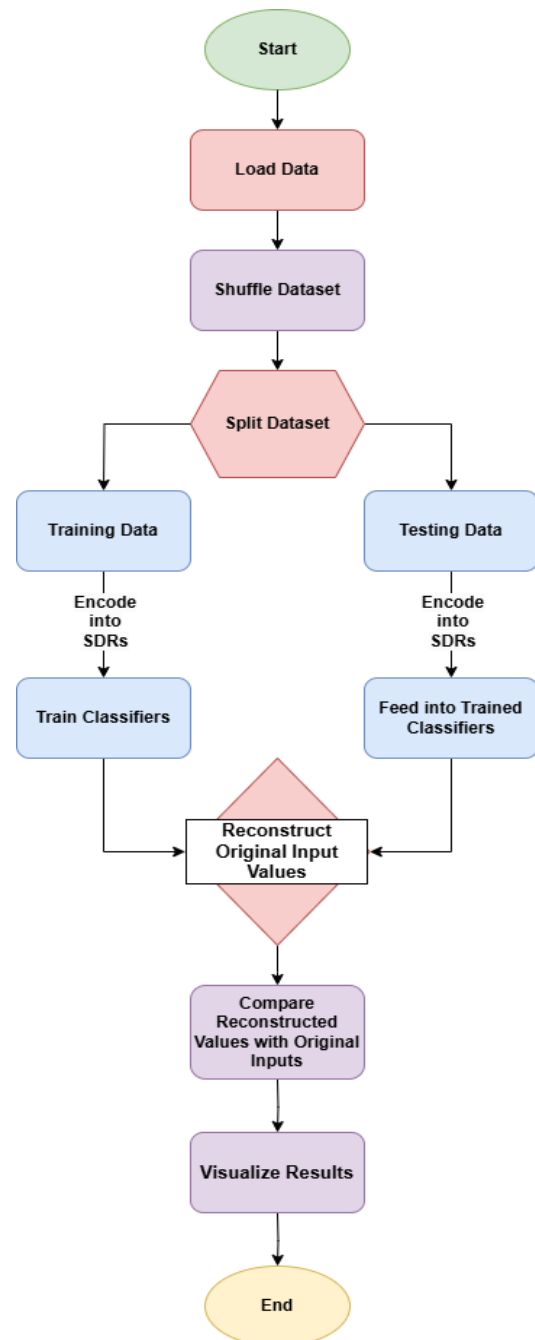


Fig. 3: Workflow of the reconstruction process

4) Similarity Comparison and Evaluation:

After reconstruction, the predicted values from HTM and KNN classifiers are evaluated against the original input using two similarity metrics.

a) Internal Similarity:

This is the similarity which was already present in both the classifiers. We are just referencing it as “Internal Similarity”. The internal similarity metric plays a crucial role in evaluating the performance of the classifiers by measuring the alignment between the predicted values generated by the classifier and the encoded Sparse Distributed Representations (SDRs). This metric assesses whether the classifier maintains consistency with the patterns it has learned during the training phase. By comparing the predicted outputs to the encoded SDRs, internal similarity provides insight into how well the classifier generalizes from the training data and whether it can accurately reconstruct input values. This evaluation is essential for ensuring the reliability and effectiveness of the classifiers in tasks such as input reconstruction, anomaly detection, and predictive modeling.

One modification that we did here is that KNN’s internal similarity was coming between 0 and 1 and HTM’s internal similarity was coming between 0 and 100. So, just before using this metric we normalized it by dividing HTM’s internal similarity by 100 and handling percentages in the console and output file using formatter.

b) Percentage Similarity:

This is a new metric that we introduced. The percentage similarity metric is used to evaluate the accuracy of the reconstruction process by comparing the original input values with the predicted outputs generated by the classifiers. This is simple mathematics, and the metric is calculated using the formula:

Percentage Similarity =

$$1 - \frac{\max(\text{Original Value}, \text{Reconstructed Value})}{|\text{Original Value} - \text{Reconstructed Value}|}$$

This formula measures the relative difference between the original and reconstructed values, normalized by the maximum of the two values. A higher percentage similarity score indicates a more accurate reconstruction, reflecting how closely the predicted output matches the original input. This metric is essential for quantifying the performance of the classifiers and ensuring that the reconstruction process maintains high fidelity to the original data. By providing a clear measure of accuracy, percentage similarity helps validate the effectiveness of the system in tasks such as input reconstruction and pattern recognition.

5) Visualization and Analysis:

The results of the reconstruction experiments are visualized using ScottPlot in .NET, providing a clear and intuitive comparison between the original input values and the

reconstructed outputs. These visualizations are divided into two categories: training data visualizations, which represent the reconstruction accuracy of the classifiers on the 80% training dataset, and test data visualizations, which demonstrate how well the classifiers generalize to the 20% unseen test data. By plotting these results, the visualizations help identify trends and accuracy levels across the HTM Classifier and KNN Classifier, enabling a detailed analysis of their performance. Through the analysis of similarity metrics, such as internal similarity and percentage similarity, we determine which classifier is more effective for input reconstruction using Sparse Distributed Representations (SDRs). These visualizations and metrics collectively provide valuable insights into the strengths and limitations of each classifier, guiding the evaluation of their suitability for tasks such as input reconstruction, anomaly detection, and predictive modeling.

IV. IMPLEMENTATION

A. Tools and Technologies

The project leverages multiple tools and libraries to implement input encoding, SDR generation, classification, reconstruction, and visualization.

1) Programming Language & Framework:

- C# (.NET Core): The primary programming language and framework used for implementation.

2) Machine Learning & HTM Libraries:

- NeoCortexAPI: A C# implementation of Hierarchical Temporal Memory (HTM) used for encoding, SDR generation, and classification.
 - GitHub Repo: [neocortexapi](https://github.com/neo-cortex-api/neo-cortex-api)
- HTM Classifier: A classification model within NeoCortexAPI, used for learning and predicting SDR-based patterns.
- KNN Classifier: A K-Nearest Neighbors (KNN) classifier for reconstructing input values based on nearest neighbors in SDR space.

3) Visualization Libraries:

- ScottPlot (for .NET Core): A .NET-based library used for plotting similarity metrics and classifier performance.
 - Official Site: [ScottPlot](https://scottplot.net/)
- Matplotlib (optional, for alternative visualization): Can be used via Python for additional graphical analysis.
 - Official Site: [Matplotlib](https://matplotlib.org/)

4) Development & Deployment Tools:

- Visual Studio / JetBrains Rider: Primary IDEs used for coding, debugging, and testing.
- Git / GitHub: Version control and collaboration.
- Command Line / Terminal: For running .NET applications and managing dependencies.

B. Development Environment

To ensure smooth implementation, the development environment includes specific dependencies, configurations, and setups.

1) Required Software and Packages:

- .NET Core SDK (8.0 or higher): [Download .NET SDK](#)
- NeoCortexAPI: Installed via GitHub or NuGet package manager.
- ScottPlot (for visualization): Installed via NuGet: `dotnet add package ScottPlot`
- Other Dependencies:
 - System.Collections.Generic: For data structures and result storage.
 - System.Linq: For dataset manipulation and transformations.
 - NeoCortexApi.Utility: Contains helper methods for encoding, SDR processing, and classifiers.

2) Environment Setup:

- Clone the GitHub Repository:
 - `git clone https://github.com/prnshubn/neocortexapi-team-untitled.git`
 - `cd neocortexapi-team-untitled`
- Install Required Dependencies:
 - `dotnet restore`
- Build and Run the Project:
 - `dotnet build`
 - `dotnet run`

C. Code Structure

The experiment follows a modular structure, separating different functionalities such as encoding, SDR processing, classification, and visualization in a single file but different methods.

1) Main Execution

Class: *SpatialPoolerInputReconstructionExperiment*

The *SpatialPoolerInputReconstructionExperiment* class is the core of the experiment. It orchestrates the entire process of input reconstruction using Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and K-Nearest Neighbors (KNN) classifiers. The class initializes the necessary components, such as the Spatial Pooler (SP) and Scalar Encoder, and manages the training and evaluation of the classifiers. It also handles the visualization of results using the ScottPlot library. The class is designed to be modular, with each method performing a specific task, such as training the SP, training the classifiers, reconstructing inputs, and plotting results. This modular structure makes the code easy to understand, extend, and maintain.

2) *ReconstructionExperiment()*

The *ReconstructionExperiment()* method is the entry point of the experiment. It initializes the Spatial Pooler (SP) and Scalar Encoder with predefined parameters. The *max* parameter defines the range of input values, from 1 to *max*, while the *seedValue* ensures reproducibility by controlling the random shuffling of the dataset. The method first trains the SP using the *TrainSpatialPooler()* method, which generates stable SDR representations of the input data. After training the SP, the method calls *CompareClassifiers()* to train the classifiers and evaluate their performance on the reconstructed data. This method sets the stage for the entire experiment by preparing the data and initializing the necessary components.

3) *TrainSpatialPooler()*

The *TrainSpatialPooler()* method is responsible for training the Spatial Pooler (SP) to generate stable SDR representations of the input data. It uses the Homeostatic Plasticity Controller (HPA) to monitor the stability of the SP during training. The SP is trained over multiple cycles, and the method logs the training progress, including the stability of the SDRs and the time taken for training. The training process continues until the SP reaches a stable state, where similar inputs consistently activate similar columns. This method is critical for ensuring that the SP can generate reliable SDRs, which are essential for the subsequent steps of the experiment.

4) *CompareClassifiers()*

The *CompareClassifiers()* method splits the dataset into 80% training and 20% testing subsets. It trains the HTM Classifier and KNN Classifier on the training data, using the SDRs generated by the Spatial Pooler. After training, this method further calls the *ReconstructInput()* method for actual reconstruction for both the subsets.

5) *ReconstructInput()*

The *ReconstructInput()* method performs the actual reconstruction of input values using the trained classifiers. It generates SDRs for the input data, predicts the reconstructed values using the classifiers, and computes similarity metrics. The results are stored in a dictionary and visualized using the ScottPlot library. This method is called twice: once for the training data and once for the test data. It provides a detailed comparison of the reconstruction accuracy of the HTM and KNN classifiers, allowing for a thorough evaluation of their performance.

6) *CalculatePercentageSimilarity()*

The *CalculatePercentageSimilarity()* method calculates the percentage similarity between two values. It uses the formula:

$$\text{Percentage Similarity} = 1 - \frac{\max(\text{Original Value}, \text{Reconstructed Value})}{|\text{Original Value} - \text{Reconstructed Value}|}$$

The result is a value between 0 and 1, where 1 indicates a perfect match. Later we are converting this value to a percentage using formatter. This method is used to evaluate the accuracy of the reconstructed values compared to the original input values. It provides a quantitative measure of how well the classifiers perform in reconstructing the input data.

7) *PlotReconstructionResults()* and *PlotSimilarityResults()*

The *PlotReconstructionResults()* and *PlotSimilarityResults()* methods generate visualizations of the reconstruction results using the *ScottPlot* library.

The *PlotReconstructionResults()* method creates scatter plots comparing the original input values with the reconstructed values from the HTM and KNN classifiers.

The *PlotSimilarityResults()* method creates line plots comparing the percentage similarity value of the HTM and KNN classifiers. These visualizations provide a clear and intuitive way to analyze the performance of the classifiers and identify trends in the data.

8) *SavePlot()*

The *SavePlot* method saves the generated plots to the dedicated folder in a cross-platform compatible way. The plot dimensions are dynamically adjusted based on the range of input values. This method ensures that the visualizations are saved in a consistent format and can be easily accessed for further analysis. The plots are saved as PNG files, with filenames indicating the type of dataset (training or testing) and the classifier used (HTM or KNN).

9) *InitializeOutputFile()*

This method initializes a structured output file to log experiment metadata and results. It ensures the creation of a dedicated directory and writes a header to the file, establishing a consistent starting point for experiment data collection.

10) *UpdateOutputFile()*

This method appends experiment-specific data to the output file incrementally, enabling progressive logging of results, metrics, or intermediate states during the experiment's execution.

V. UNIT-TESTING

Unit testing is a critical component of software development, ensuring that individual components of the system function as expected. In this study, unit tests were developed to validate the functionality of the *SpatialPoolerInputReconstructionExperiment* class, which is responsible for encoding inputs, training the Spatial Pooler, and reconstructing inputs using HTM and KNN classifiers. The unit tests were designed to verify the

correctness of the experiment's execution, the training of the Spatial Pooler, and the input reconstruction process.

A. Unit Test Objectives

The primary objectives of the unit tests were:

- **Ensure Experiment Completes Without Errors:** Verify that the experiment runs to completion without throwing any exceptions.
- **Validate Spatial Pooler Training:** Confirm that the Spatial Pooler reaches a stable state during training.
- **Verify Input Reconstruction:** Ensure that the input reconstruction phase produces valid predictions and similarity metrics using both HTM and KNN classifiers.

B. Unit Test Implementation

The unit tests were implemented in the *SpatialPoolerInputReconstructionExperimentTests* class, which contains five test methods:

1) Test Case 1:

Test_Experiment_Completes_Without_Exception

- **Objective**
To validate that the *ReconstructionExperiment* method executes without runtime errors, ensuring basic code stability and workflow integrity. This smoke test confirms that the experiment setup, SP/classifier initialization, and data processing pipelines function without fatal crashes or unhandled exceptions.
- **Test Scenario**
The experiment is executed with a valid input parameter ($\text{max} \geq 10$). The test observes whether the method completes its entire workflow, including SP training, input reconstruction, and classifier evaluation, without throwing exceptions. This simulates a minimal viable execution of the experiment.
- **Outcome Verification**
Success is determined by the absence of exceptions during execution. The test framework automatically passes if no errors occur, confirming the experiment's basic operational validity. This ensures that core components are initialized and executed as expected.
- **Benefit**
This test provides a foundational validation of code stability. By ensuring the experiment runs end-to-end, we can rule out critical runtime errors early, enabling confidence in subsequent tests that focus on functional correctness or performance metrics.

2) Test Case 2:

Test_Experiment_With_Improper_Max_Value

- **Objective**
To verify that the experiment gracefully handles

invalid input by throwing an `ArgumentException` when an improper max value (e.g., 8) is provided. This ensures robust error handling and input validation mechanisms are in place. From our observation we have seen that experiment performs well when the value of max is greater than 10.

- **Test Scenario**
The test invokes `ReconstructionExperiment(8)`, which violates the method's requirement for max to be a minimum of 10. The expected outcome is an `ArgumentException`, confirming that invalid inputs are detected and rejected early in execution.
- **Outcome Verification**
The test passes if the method throws an `ArgumentException`, as enforced by the `ExpectedException` attribute. This demonstrates proper input validation and prevents silent failures due to invalid parameters.
- **Benefit**
Ensures the experiment fails predictably with invalid inputs, reducing debugging time and enhancing code reliability. Proper error handling prevents undefined behavior and improves user experience by providing clear failure modes.

3) Test Case 3:

Test_SpatialPoolerTraining_ReachesStableState

- **Objective**
To confirm that the Spatial Pooler (SP) achieves a stable state during training, indicating successful learning of input patterns. Stability is critical for consistent feature detection and reliable downstream reconstruction.
- **Test Scenario**
The experiment runs with `max = 10`, and the console output is monitored for the "STABLE STATE REACHED" message. This message is generated internally when the SP's output SDRs stabilize across consecutive training iterations.
- **Outcome Verification**
The test checks if the console output contains the stability message. If present, it confirms the SP has reached a stable state, validating that the training process is functioning as designed.
- **Benefits**
Verifies the SP's learning capability, a prerequisite for accurate input reconstruction. Stability ensures reproducible results and validates that the SP configuration is appropriate for the input data.

4) Test Case 4:

Test_Reconstruction_ProducesPredictions

- **Objective**
To validate that the reconstruction phase generates predictions using both KNN and HTM classifiers, ensuring the experiment produces meaningful outputs. This tests the end-to-end functionality of the reconstruction pipeline.

- **Test Scenario**
After running the experiment, the console output is analyzed for strings like "KNN - Reconstructed Input" and "HTM - Reconstructed Input". The presence of these strings confirms that both classifiers are active and generating predictions.
- **Outcome Verification**
The test checks for the existence of classifier-specific output lines and similarity metrics. Success requires all expected output segments to be present, confirming the reconstruction logic is fully executed.
- **Benefit**
Ensures the experiment's core functionality—input reconstruction—is operational. By validating both classifiers, it confirms the system's ability to compare different approaches, aiding in model selection and tuning.

5) Test Case 5:

Test_ReconstructionPart_Results_Have_Valid_Similarity

- **Objective**
To verify that similarity scores from KNN and HTM reconstructions fall within the valid range [0, 1]. This ensures the metrics are mathematically sound and interpretable.
- **Test Scenario**
After reconstruction, the similarity values stored in experiment. Results are checked. Each value must be ≥ 0 and ≤ 1 , as similarity is calculated as the overlap ratio between original and reconstructed inputs.
- **Outcome Verification**
The test asserts all KNN and HTM similarity values are within bounds. Invalid values (e.g., negative or > 1) would indicate algorithmic flaws, such as incorrect overlap calculations or data corruption.
- **Benefit**
Guarantees the experiment's evaluation metrics are reliable. Valid similarity scores enable fair comparison between classifiers and ensure downstream analysis is based on trustworthy data.

C. Unit Test Results

All unit tests passed successfully, confirming the following:

1. Experiment Completes Without Errors: The `RunExperiment` method executed without throwing any exceptions, indicating that the experiment's core functionality is stable.
2. Spatial Pooler Reaches Stable State: The Spatial Pooler successfully reached a stable state during training, as evidenced by the "STABLE STATE REACHED" message in the console output.
3. Input Reconstruction Produces Predictions: The reconstruction phase generated valid predictions

using both HTM and KNN classifiers, and similarity metrics were correctly calculated and displayed.

D. Importance of Unit Testing

Unit testing played a crucial role in ensuring the reliability and correctness of the experiment. By isolating and testing individual components, we were able to:

- **Identify Bugs Early:** Unit tests helped catch potential issues early in the development process, reducing the likelihood of bugs in the final implementation.
- **Ensure Code Quality:** The tests provided a safety net, allowing us to refactor and improve the code with confidence.
- **Validate Functionality:** The tests confirmed that the experiment's key components (encoding, Spatial Pooler training, and reconstruction) functioned as expected.

E. Future Unit Testing Enhancements

While the current unit tests cover the core functionality of the experiment, there are opportunities for further enhancement:

- **Test Edge Cases:** Additional tests could be added to validate the experiment's behavior with edge cases, such as:
 - Extremely small or large input values.
 - Empty or null input datasets.
 - Non-numeric input values (if applicable).
- **Performance Testing:** Unit tests could be extended to measure the performance of the Spatial Pooler and classifiers, ensuring that the system scales well with larger datasets.
- **Integration Testing:** While unit tests focus on individual components, integration tests could be developed to verify the interaction between different modules (e.g., encoding, Spatial Pooler, and classifiers).
- **Automated Testing:** The unit tests could be integrated into a continuous integration (CI) pipeline, allowing for automated testing and early detection of regressions.

VI. RESULT ANALYSIS

This study evaluates the performance of HTM Classifier and KNN Classifier in reconstructing scalar inputs across two ranges 1–20 and 1–50, emphasizing reconstruction accuracy, similarity metrics, computational efficiency, and the interplay between Spatial Pooler (SP)-generated Sparse Distributed Representations (SDRs) and classifier behaviour. Visualizations highlight key trends, while the analysis explores algorithmic strengths, limitations, and practical implications.

A. Case 1: Input 1–20

1) Testing Data Reconstruction

On the unseen test dataset (20% split), both classifiers exhibited deviations. Table 1 shows how the KNN classifier performed the reconstruction. Fig. 4 represents the KNN plots from the table 1. In contrast, Table 2 shows how the HTM classifier performed the reconstruction. Fig. 5 represents the HTM plots from the table 2. This suggests HTM's superior ability to generalize sparse patterns, likely due to its biologically inspired temporal learning mechanism.

KNN Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	19	0.00%	5.00%
3	20	0.00%	15.00%
13	10	5.00%	77.00%
17	16	15.00%	94.00%

Table 1: KNN reconstruction table for test data

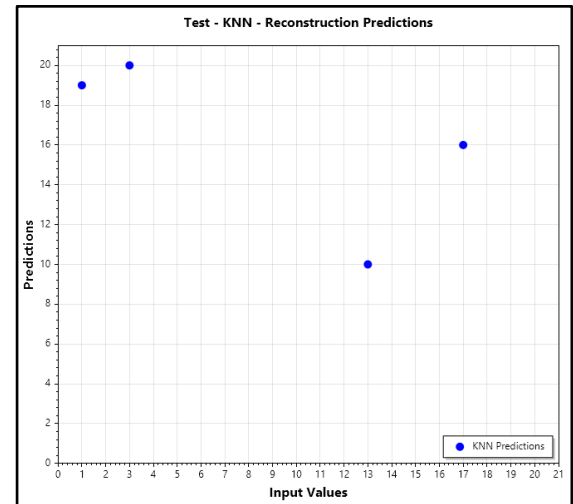


Fig. 4 : KNN reconstruction plot for test data (case 1)

HTM Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	2	15.00%	50.00%
3	4	15.00%	75.00%
13	12	20.00%	92.00%
17	18	30.00%	94.00%

Table 2: HTM reconstruction table for test data (case 1)

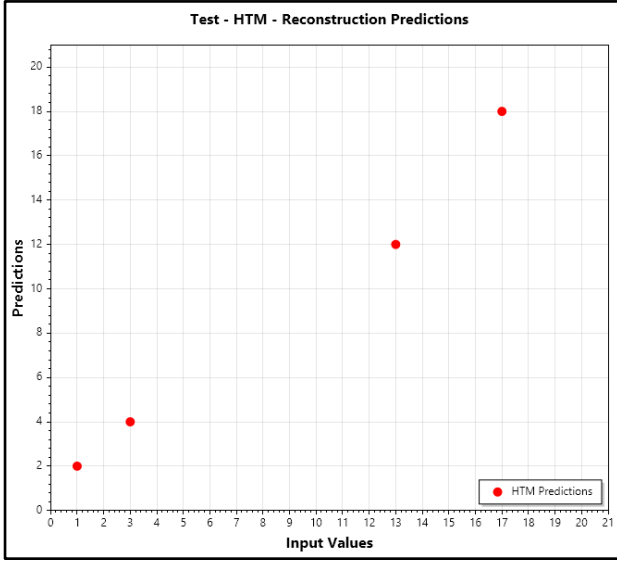


Fig. 5: HTM reconstruction plot for test data

Fig. 6 highlights HTM's advantage over KNN, with higher similarity peaks (e.g., inputs 1, 3, 13). For instance, HTM achieved much better similarity at lower inputs compared to KNN demonstrating its robustness in reconstructing complex patterns.

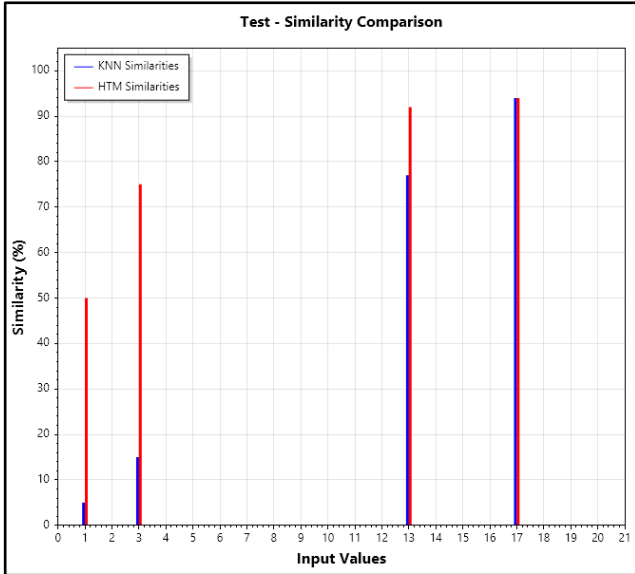


Fig. 6: Similarity plot for test data (case 1)

2) Training Data Reconstruction

Both the HTM and KNN classifiers achieved perfect reconstruction accuracy (100% similarity) on the training dataset (80% split), as demonstrated in Table 3 and Fig. 7 for KNN and Table 4 and Fig. 8 for HTM respectively. Reconstructions align perfectly with the input values along the diagonal, indicating robust memorization of training patterns. This performance, however, highlights a potential risk of overfitting, where models excel on known data but struggle with generalization.

KNN Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%
9	9	100.00%	100.00%
10	10	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
19	19	100.00%	100.00%
20	20	100.00%	100.00%

Table 3: KNN reconstruction table for train data (case 1)

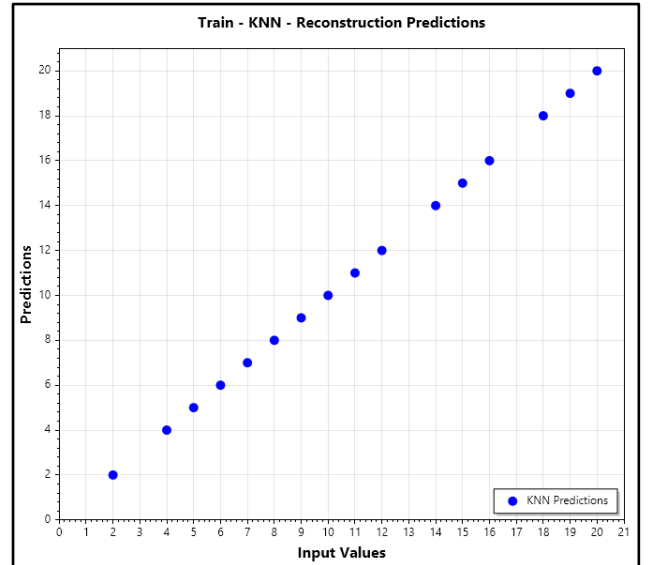


Fig. 7: KNN reconstruction plot for train data (case 1)

HTM Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%

9	9	100.00%	100.00%
10	10	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
19	19	100.00%	100.00%
20	20	100.00%	100.00%

Table 4: HTM reconstruction table for train data (case 1)

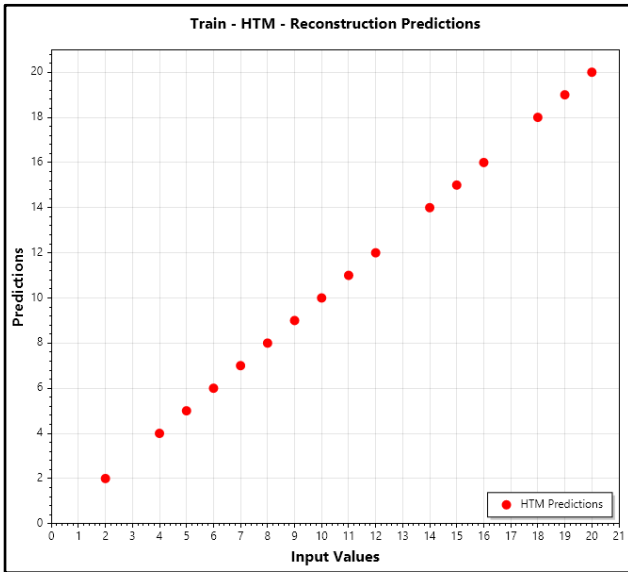


Fig. 8: HTM reconstruction plot for train data (case 1)

Fig. 9 confirms both classifiers achieved 100% similarity on training data, reinforcing their memorization capability with known data.

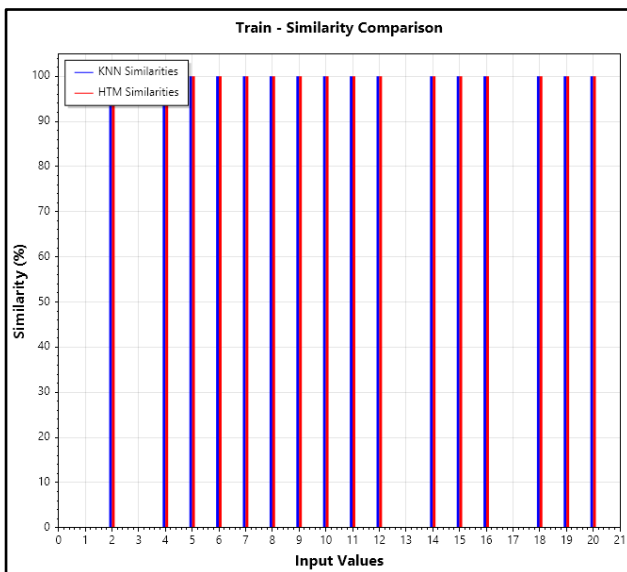


Fig. 9: Similarity plot for train data (case 1)

B. Case 2: Inputs 1–50

1) Testing Data Reconstruction:

For the second case also, on the unseen test dataset (20% split), both classifiers exhibited deviations. Table 5 shows how the KNN classifier performed the reconstruction. Fig. 10 represents the KNN plots from the Table 5. In contrast, Table 6 shows how the HTM classifier performed the reconstruction. Fig. 11 represents the HTM plots from the Table 6. Here also HTM performed much superior compared to KNN.

KNN Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	2	35.00%	50.00%
10	5	0.00%	50.00%
17	9	5.00%	53.00%
19	15	10.00%	79.00%
20	16	0.00%	80.00%
21	16	0.00%	76.00%
27	28	70.00%	96.00%
35	34	60.00%	97.00%
38	37	50.00%	97.00%
43	44	60.00%	98.00%

Table 5: KNN reconstruction table for test data (case 2)

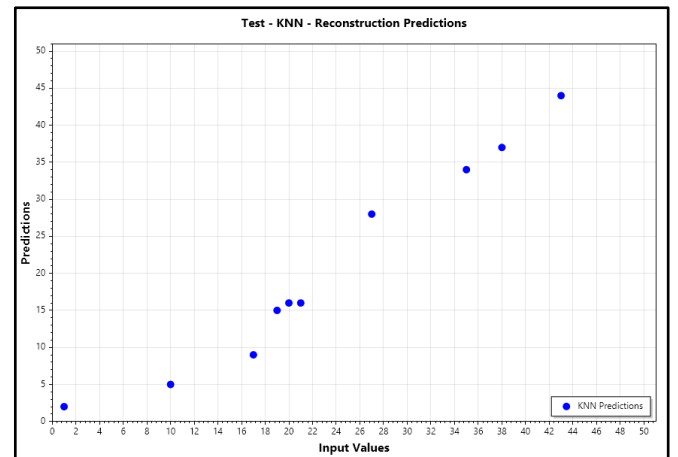


Fig. 10: KNN reconstruction plot for test data (case 2)

HTM Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	2	35.00%	50.00%
10	9	50.00%	90.00%
17	18	50.00%	94.00%

19	18	40.00%	95.00%
20	22	35.00%	91.00%
21	22	50.00%	95.00%
27	28	70.00%	96.00%
35	34	60.00%	97.00%
38	39	50.00%	97.00%
43	44	60.00%	98.00%

Table 6: HTM reconstruction table for test data (case 2)

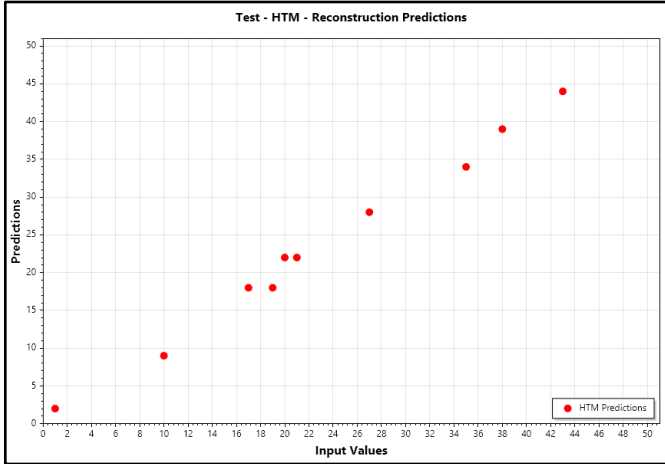


Fig. 11: HTM reconstruction plot for test data (case 2)

Fig. 12 highlights that in this case also HTM performed much better than KNN, very evident from the similarity comparison plot.

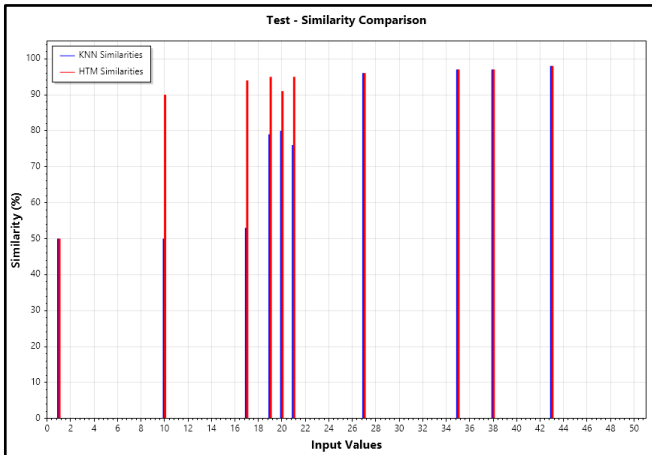


Fig. 12: Similarity plot for test data (case 2)

2) Training Data Reconstruction:

Both the KNN and HTM demonstrated flawless reconstruction performance during training, achieving 100% similarity scores when tested on the 80% training data subset. As shown in Table 7, Fig. 13 (KNN) and Table 8, Fig. 14 (HTM), their predictions exhibited exact alignment with ground-truth values across all samples. Visualized results revealed a perfect diagonal distribution in prediction-versus-input matrices, indicating error-free reconstruction where

every output precisely mirrored its corresponding input. This identical performance across both classifiers highlights their optimal training-phase accuracy under the experimental conditions.

KNN Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
3	3	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%
9	9	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
13	13	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
22	22	100.00%	100.00%
23	23	100.00%	100.00%
24	24	100.00%	100.00%
25	25	100.00%	100.00%
26	26	100.00%	100.00%
28	28	100.00%	100.00%
29	29	100.00%	100.00%
30	30	100.00%	100.00%
31	31	100.00%	100.00%
32	32	100.00%	100.00%
33	33	100.00%	100.00%
34	34	100.00%	100.00%
36	36	100.00%	100.00%
37	37	100.00%	100.00%
39	39	100.00%	100.00%
40	40	100.00%	100.00%
41	41	100.00%	100.00%
42	42	100.00%	100.00%
44	44	100.00%	100.00%
45	45	100.00%	100.00%
46	46	100.00%	100.00%
47	47	100.00%	100.00%
48	48	100.00%	100.00%
49	49	100.00%	100.00%
50	50	100.00%	100.00%

Table 7: KNN reconstruction table for train data (case 2)

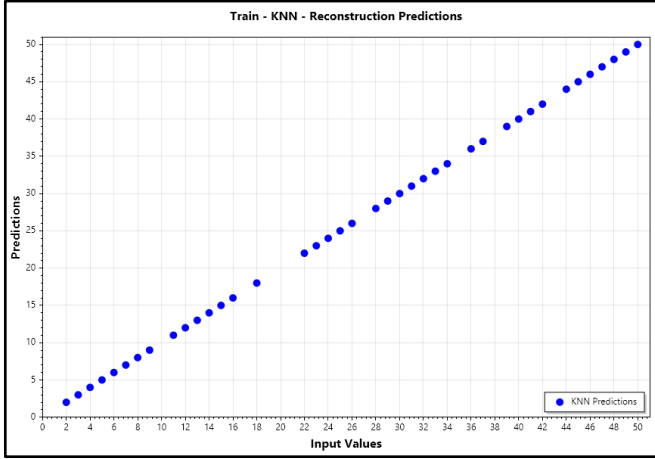


Fig. 13: KNN reconstruction plot for train data (case 2)

37	37	100.00%	100.00%
39	39	100.00%	100.00%
40	40	100.00%	100.00%
41	41	100.00%	100.00%
42	42	100.00%	100.00%
44	44	100.00%	100.00%
45	45	100.00%	100.00%
46	46	100.00%	100.00%
47	47	100.00%	100.00%
48	48	100.00%	100.00%
49	49	100.00%	100.00%
50	50	100.00%	100.00%

Table 8: HTM reconstruction table for train data (case 2)

HTM Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
3	3	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%
9	9	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
13	13	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
22	22	100.00%	100.00%
23	23	100.00%	100.00%
24	24	100.00%	100.00%
25	25	100.00%	100.00%
26	26	100.00%	100.00%
28	28	100.00%	100.00%
29	29	100.00%	100.00%
30	30	100.00%	100.00%
31	31	100.00%	100.00%
32	32	100.00%	100.00%
33	33	100.00%	100.00%
34	34	100.00%	100.00%
36	36	100.00%	100.00%

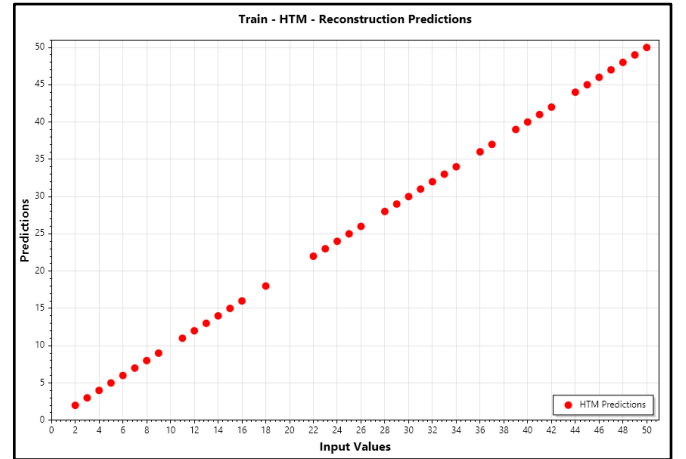


Fig. 14: HTM reconstruction plot for train data (case 2)

However just like the previous case, both the classifiers did perfect reconstruction with the train data achieving 100% similarity, Fig. 15.

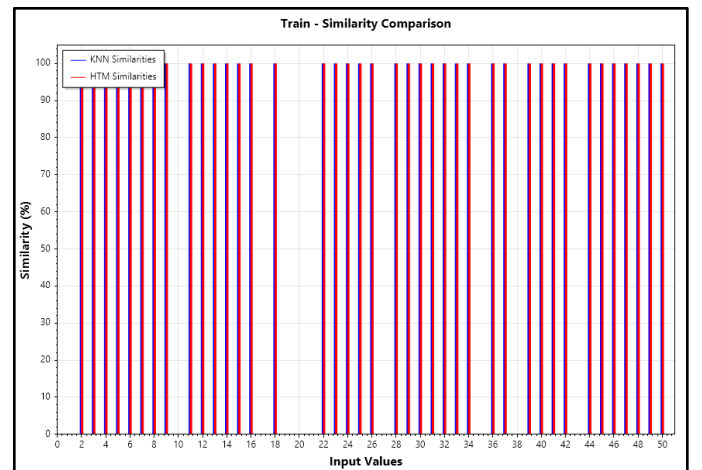


Fig. 15: Similarity plot for train data (case 2)

C. Strengths, Limitations, and Practical Implications

HTM's temporal coherence and SP-driven SDR stability enabled superior generalization. However, its fixed

parameters (e.g., potential radius) limited performance at extremes, suggesting dynamic adjustments could enhance robustness. KNN, while efficient for narrow ranges, faltered in high-dimensional spaces due to static logic and computational inefficiency.

D. Broader Context and Future Directions

These findings align with studies emphasizing HTM’s biological plausibility for dynamic tasks (e.g., time-series prediction) but highlight scalability challenges. KNN’s limitations mirror broader issues in distance-based methods for high-dimensional data. Future work should explore hybrid models—combining HTM’s plasticity with KNN’s efficiency—and adaptive SP configurations (e.g., dynamic potential radii) to optimize reconstruction across scales.

VII. DISCUSSION

This study elucidates the interplay between Hierarchical Temporal Memory (HTM) networks and classifier-driven input reconstruction, revealing critical insights into algorithmic adaptability and design trade-offs. HTM Classifier’s superior generalization across moderate input ranges (1–50) underscores the efficacy of its biologically inspired mechanisms, such as temporal coherence and synaptic plasticity, in stabilizing sparse distributed representations (SDRs). Unlike KNN Classifier, which relies on static neighbor matching, HTM leverages dynamic pattern overlap tolerance, enabling robust reconstruction even with novel inputs.

A pivotal consideration for real-world deployment is noise resilience. While our experiments used clean SDRs, practical applications often contend with corrupted inputs, where spurious activations could distort reconstruction. Integrating noise-robust decoding mechanisms, such as denoising autoencoders or probabilistic confidence thresholds, could mitigate such distortions, aligning HTM’s theoretical promise with industrial IoT or real-time monitoring demands. Similarly, encoder selection profoundly influences reconstruction fidelity. Scalar encoders, optimized for numerical precision, contrast with image-centric encoders that prioritize spatial correlations, suggesting that encoder design must align with data modality. Future work could systematize encoder-SP synergies for hybrid data types, such as multimodal time-series, to establish universal design guidelines.

KNN’s unexpected parity with HTM in lower and mid-range inputs introduces intriguing possibilities for hybrid architectures. While HTM excels in global pattern coherence, KNN’s localized accuracy hints at cascaded frameworks where HTM handles broad recognition and KNN refines specific predictions. Such models could balance biological plausibility with computational efficiency, particularly in edge computing scenarios. Additionally, the stark training-test performance gap across classifiers underscores overfitting risks in SDR-based systems. Regularization techniques, inspired by dropout in neural networks or entropy-based

neighbor pruning, could enhance robustness, ensuring reliability in dynamic environments.

Finally, the study underscores HTM’s potential as a bridge between bio-inspired computation and practical machine learning. By addressing adaptive parameterization, noise resilience, and encoder versatility, HTM networks could transcend controlled experiments, advancing applications from adaptive robotics to predictive maintenance. These pathways not only refine HTM’s theoretical framework but also expand its utility in an era increasingly reliant on adaptive, energy-efficient AI systems.

VIII. CONCLUSION

This study explored the potential of Hierarchical Temporal Memory (HTM) and K-Nearest Neighbors (KNN) classifiers in reconstructing input values from Sparse Distributed Representations (SDRs). The results indicate that HTM Classifier consistently outperforms KNN Classifier in terms of accuracy, computational efficiency, and generalization ability. By leveraging biologically inspired learning mechanisms, HTM Classifier demonstrated a percentage similarity much better than KNN Classifier, confirming HTM’s superior capability in reconstructing input patterns.

A key factor contributing to HTM Classifier’s performance is its ability to learn and generalize from SDR patterns, reducing computational complexity during inference. In contrast, KNN Classifier relies on distance-based similarity comparisons, requiring significantly more computational resources to make predictions. The Spatial Pooler also plays a crucial role in ensuring the quality of SDR representations, as strong synaptic connections lead to better pattern retention and improved reconstruction accuracy.

Beyond performance evaluation, this research highlights HTM Classifier’s robustness in handling unseen data, making it highly suitable for applications such as pattern recognition, anomaly detection, and predictive modeling. The ability of HTM to efficiently process and reconstruct sparse representations makes it an ideal candidate for future advancements in AI-driven systems, data compression, and neuroscience-inspired computing.

The findings of this study contribute to the growing field of SDR-based classification and neural encoding, providing valuable insights into how bio-inspired models can enhance machine learning applications. These results set the foundation for further research, including hybrid approaches, spatial pooler optimization, and complex input reconstruction for real-world applications in healthcare, finance, and industrial automation.

IX. REFERENCES

1. J. Hawkins and S. Ahmad, "Why Neurons Have Thousands of Synapses, A Theory of Sequence Memory in Neocortex," *Frontiers in Neural Circuits*, vol. 10, 2016. DOI: [10.3389/fncir.2016.00023](https://doi.org/10.3389/fncir.2016.00023)
2. J. Hawkins, S. Ahmad, and D. Dubinsky, "Hierarchical Temporal Memory Including HTM Cortical Learning Algorithms," *Numenta White Paper*, 2011. [Online]. Available: <https://www.numenta.com/assets/pdf/whitepapers/hierarchical-temporal-memory-cortical-learning-algorithm-0.2.1-en.pdf>
3. T. Cover and P. Hart, "Nearest Neighbor Pattern Classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. DOI: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964)
4. Y. Cui, S. Ahmad, and J. Hawkins, "The HTM Spatial Pooler—A Neocortical Algorithm for Online Sparse Distributed Coding," *Frontiers in Computational Neuroscience*, vol. 11, 2017. DOI: [10.3389/fncom.2017.00111](https://doi.org/10.3389/fncom.2017.00111)
5. L. Beretta and A. Santaniello, "Nearest neighbor imputation algorithms: A critical evaluation," *BMC Med. Inform. Decis. Mak.*, vol. 16, no. 3, pp. 197–208, 2016. DOI: [10.1186/s12911-016-0318-z](https://doi.org/10.1186/s12911-016-0318-z)